Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

ACP vs MCP: two protocols for AI agents

---

1. What is ACP?

ACP (Agent Client Protocol) is an open protocol created by Zed Industries in August 2025 (Apache-licensed and backed by JetBrains) that standardizes communication between a **code editor / IDE** and a **coding agent** (for example Claude Code, Gemini CLI or Codex).

The problem it solves is combinatorial: with N editors and M agents and no standard, every pair needs its own integration, i.e. $N \cdot M$ integrations. With ACP, each editor implements the protocol once and so does each agent, leaving $N + M$ integrations. It is the same idea the *Language Server Protocol* (LSP) applied to programming languages.

ACP defines the things an editor cares about: showing *diffs*, asking the user for approval before running actions, streaming session progress, and following what the agent is editing.

Watch out for the acronym: three different protocols are called ACP.

- **Agent Client Protocol** (Zed/JetBrains): editor ↔ agent. *The subject of this document.*
- **Agent Communication Protocol** (IBM/BeeAI): agent ↔ agent. Merged into A2A.
- **Agentic Commerce Protocol** (OpenAI/Stripe): AI-driven purchases. Unrelated.

2. What is MCP?

MCP (Model Context Protocol) is an open standard published by Anthropic in late 2024 that connects an **AI agent or application** to external **tools and data sources**: GitHub, databases, the file system, Slack, etc. Each integration is packaged as an *MCP server* that exposes *tools*, *resources* and *prompts* (prompt templates).

3. Where each one lives

The key idea is that **they do not compete: they are different layers of the same stack**. ACP sits to the *left* of the agent (towards the user) and MCP to the *right* (towards the tools).

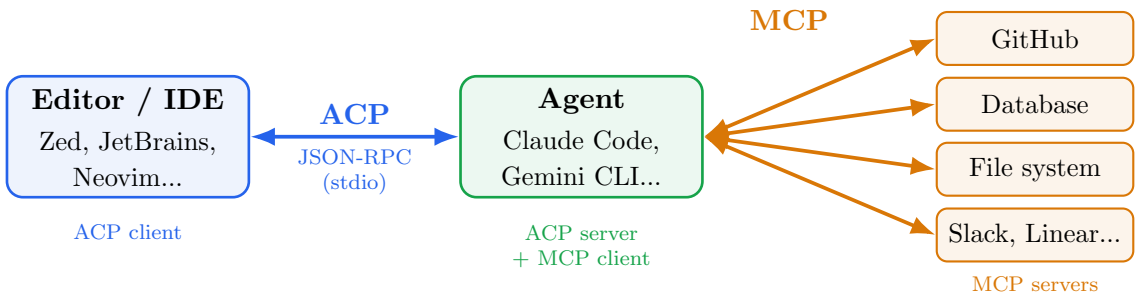


Figure 1: ACP connects the editor to the agent; MCP connects the agent to its tools.

A useful detail to tell them apart: **the roles are reversed**. In ACP the agent is the *server* (the editor is the client); in MCP the agent is the *client* (the tool is the server). The same agent speaks both protocols at once, in the same session.

4. Side-by-side comparison

	ACP (Agent Client Protocol)	MCP (Model Context Protocol)
Connects	Editor/IDE ↔ agent	Agent ↔ tools and data
Origin	Zed Industries, August 2025	Anthropic, November 2024
Client	The editor	The agent / AI app
Server	The agent	The MCP server (tool wrapper)
Transport	JSON-RPC 2.0, typically over `stdio`	JSON-RPC 2.0, over `stdio` or HTTP
Standardizes	Sessions, diffs, permissions, progress, agent UI	Tools, resources and prompts
Question it answers	How do I show and control the agent from my editor?	How do I give the agent access to my systems?
Analogy	LSP, but for agents	"USB-C" for AI tools

Table 1: Main differences between ACP and MCP.

5. A typical flow using both protocols

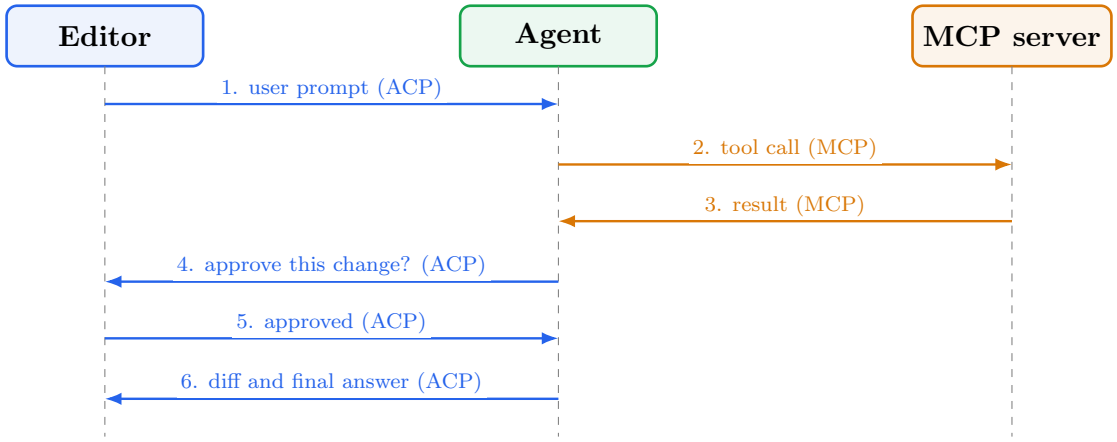


Figure 2: The user talks to the agent via ACP; the agent uses its tools via MCP.

6. When does each one matter?

- **You are building an editor or IDE** and want to support many agents without integrating each one ⇒ implement **ACP**.
- **You are building an agent** and want it to run inside several editors ⇒ expose **ACP**; to give it access to tools, act as an **MCP** client.
- **You want any agent to use your system** (your API, your database, your internal tool) ⇒ publish an **MCP** server.

Conclusion: "ACP vs MCP" is a false dichotomy. MCP gives the agent *hands* (access to tools and data); ACP gives it a *face* inside the editor (interface, permissions and diffs). A modern coding agent usually uses both at the same time.
